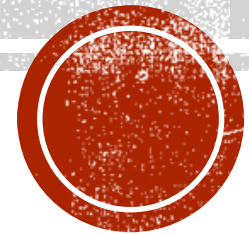


INTRODUCTION TO RECONFIGURABLE COMPUTING

CS G553: Reconfigurable computing

Dr. Meetha V Shenoy

Lect-7



FPGA Design Flow

CAD
Tools

Design Entry

Schematic editor, HDL, FSM
Editor

Functional
Simulation

Check correctness using test
patterns

Synthesis

Translate to Boolean functions and
Netlist formats → EDIF, XNF
then LUT based technology
mapping

Place and
Route

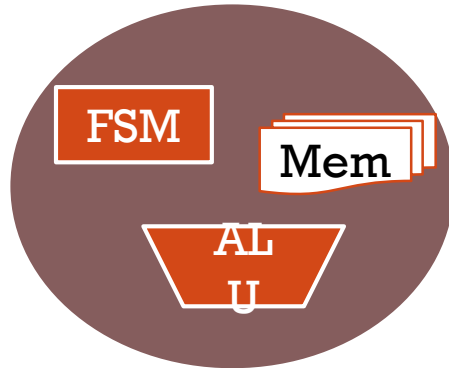
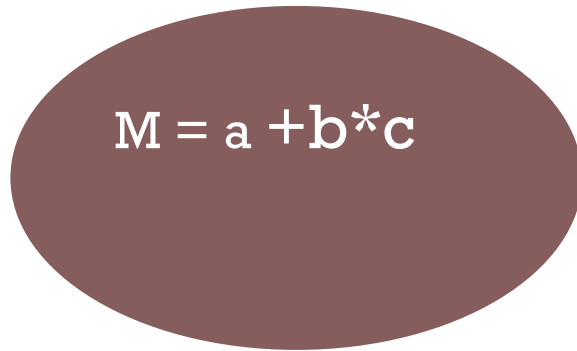
Operators should be placed and
connected

Configurati
on
Bitstream

Description of all the bits used to
configure LUTs, Interconnects, state of
Mux and I/O

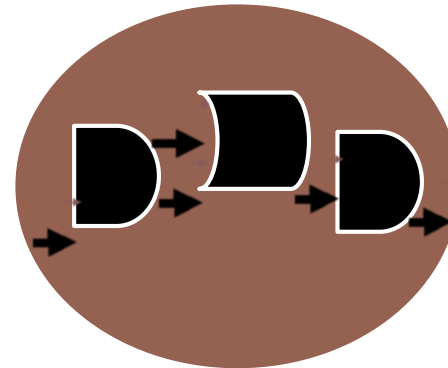
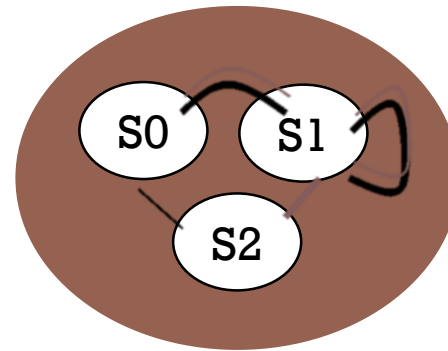


TAXONOMY OF SYNTHESIS

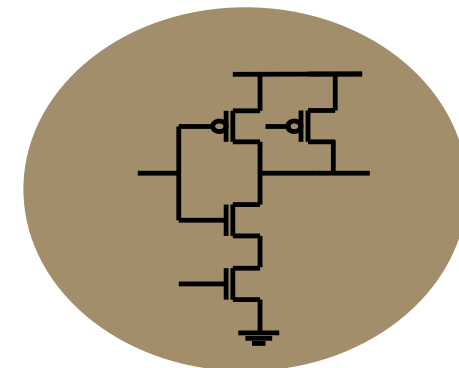
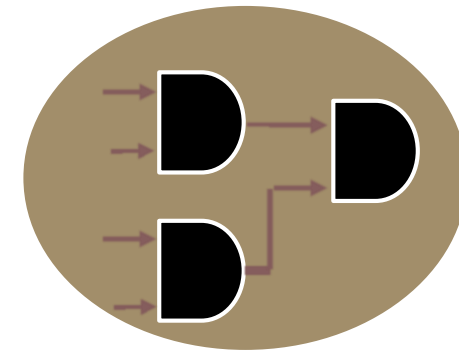


Architectural Synthesis

RC@Meetha V Shenoy



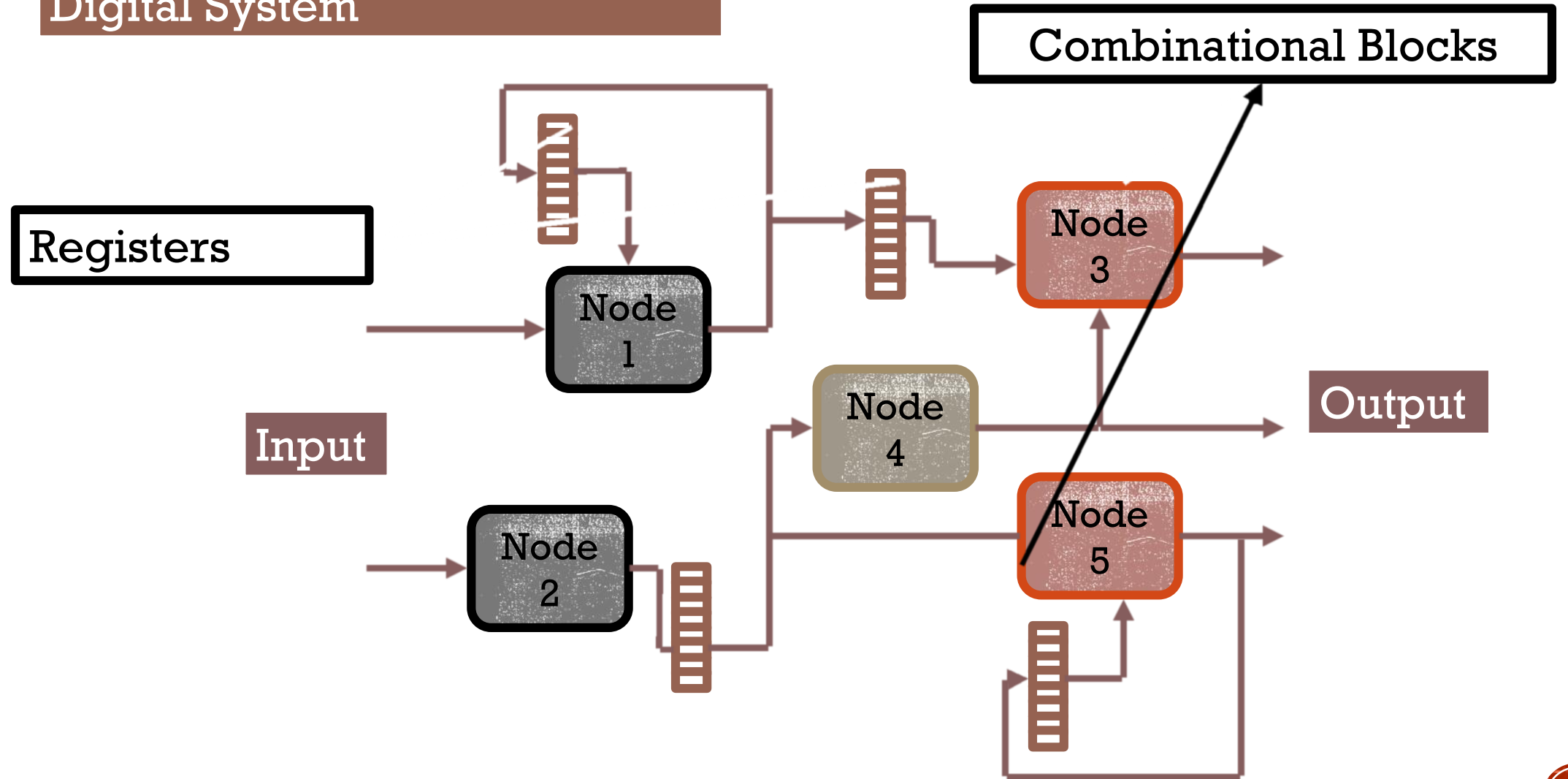
Logic Synthesis



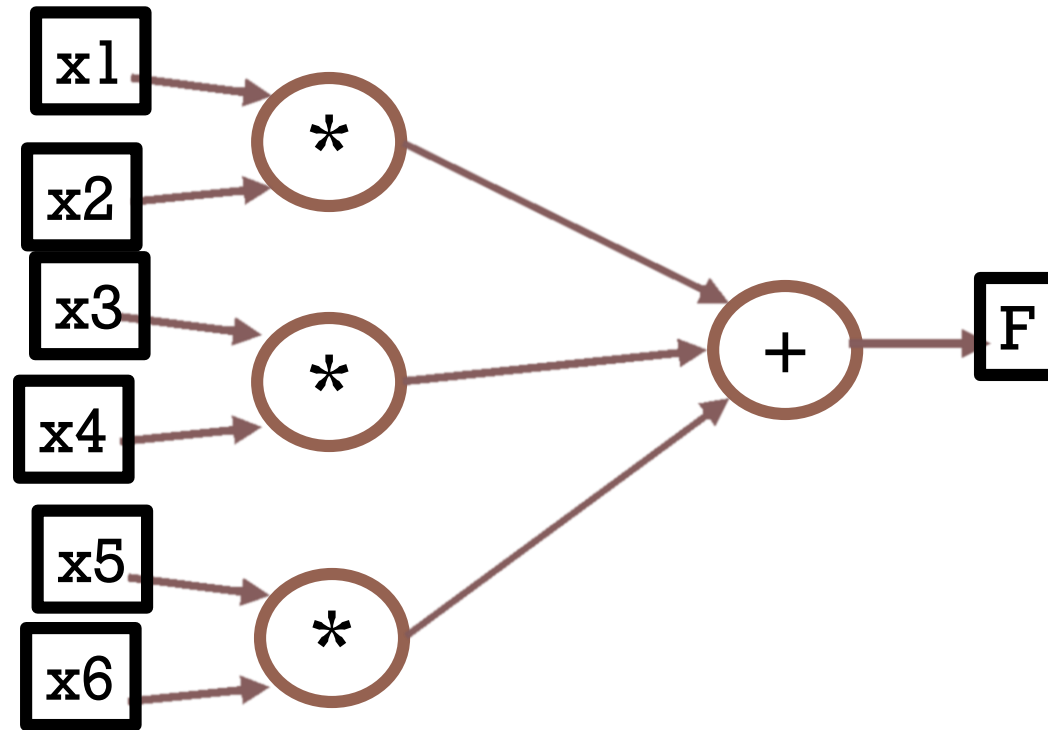
Circuit Synthesis



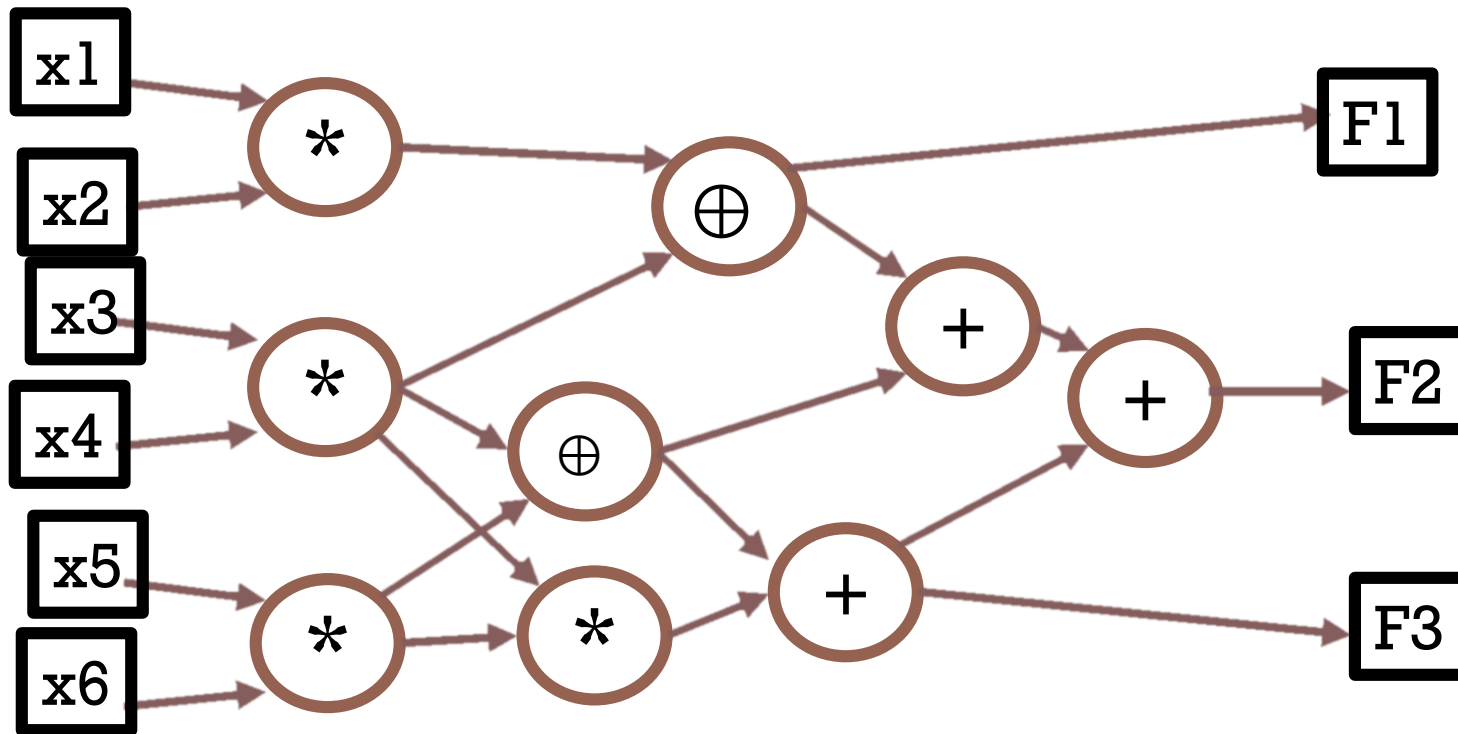
Digital System



TWO LEVEL LOGIC



MULTI-LEVEL LOGIC



Node Manipulation

Goal is to generate an optimized network with

Less number of gates

Lowest depth of gates



Node Manipulation

Transformations

Decomposition

Extraction

Factoring

Substitution

Collapsing



Node Manipulation

Decomposition

Takes a single Boolean function and replace it with a collection of new expressions.

Boolean function $f(X)$ is decomposable if we can find a function $g(X)$ such that

$$f(X) = f'(g(X), X).$$



Node Manipulation

Decomposition

$$f = a\bar{b}\bar{c} + a\bar{b}d + \bar{a}c\bar{d} + bcd$$

12 literals

2 levels

$$f = XY + \bar{X}\bar{Y}$$

$$X = a\bar{b} \qquad Y = \bar{c} + d$$

8 literals

3 levels



Node Manipulation

Extraction

Used to identify common intermediate sub-functions of a set of given functions in order to avoid redundancy.

$$f = (a + bc)d + e \qquad g = (a + bc)d\bar{e}$$

$$f = x + e \qquad g = x\bar{e}$$

$$x = (a + bc)d$$



Node Manipulation

Extraction

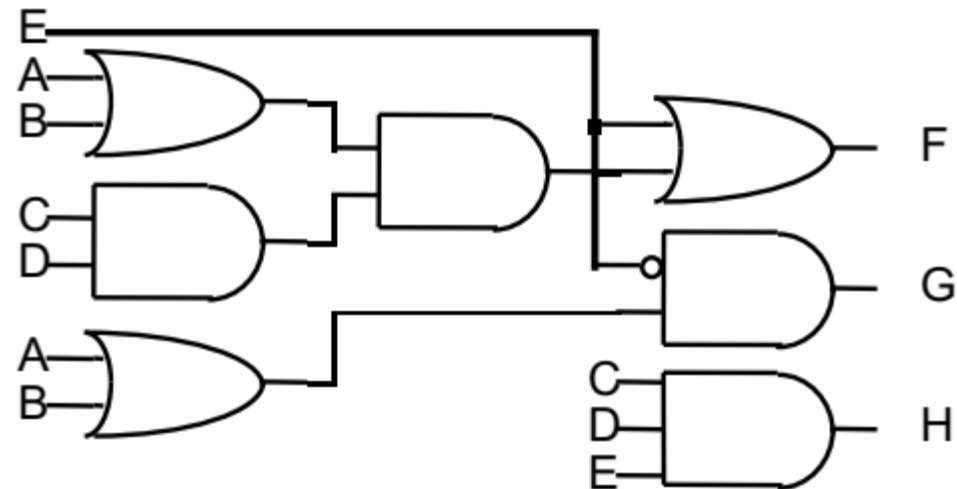
$$f = (a + b) c d + e$$

11 literals

$$g = (a + b) \bar{e}$$

8 gates

$$h = c d e$$



Before Extraction



Node Manipulation

$f = (a + b) c d + e$

$$f = (a + b) c d + e$$

$$g = (a + b) \bar{e}$$

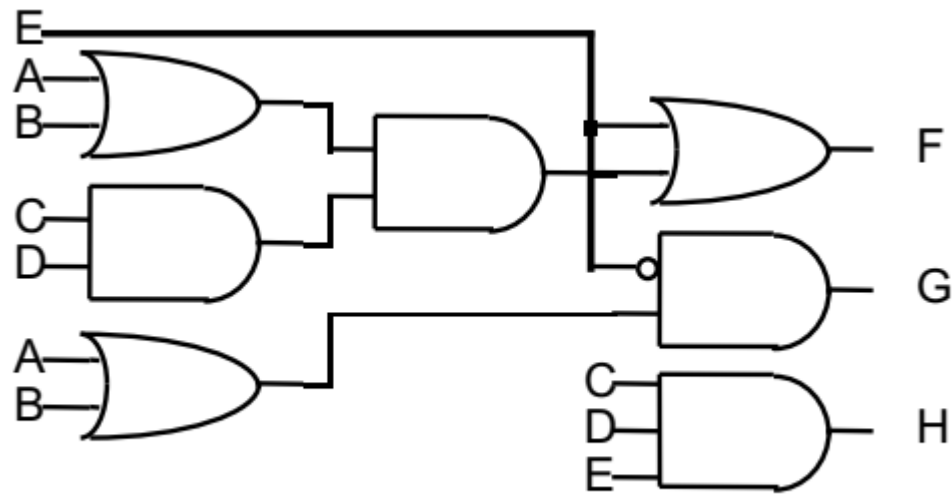
$$f = xy + e$$

$$h = c d e$$

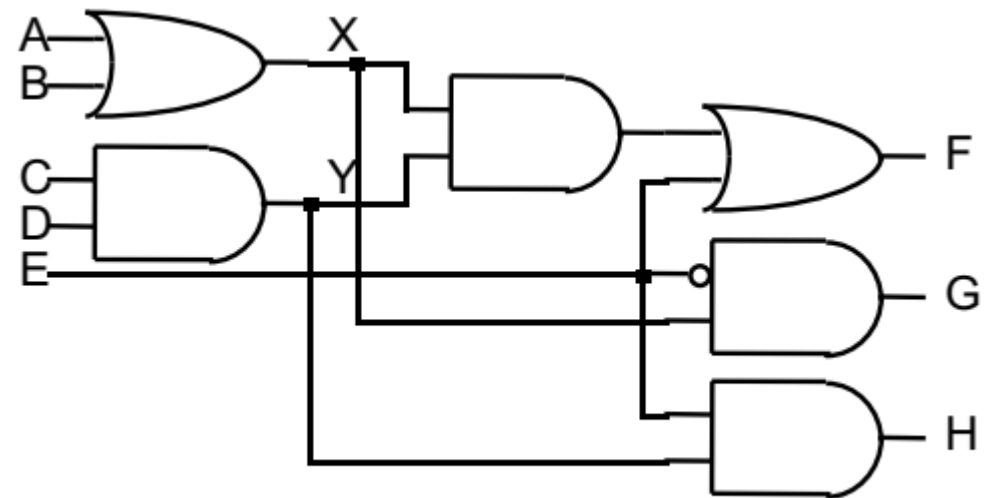
$$g = x\bar{e} \quad h = ye$$

11 literals

7 gates



Before Extraction



After Extraction

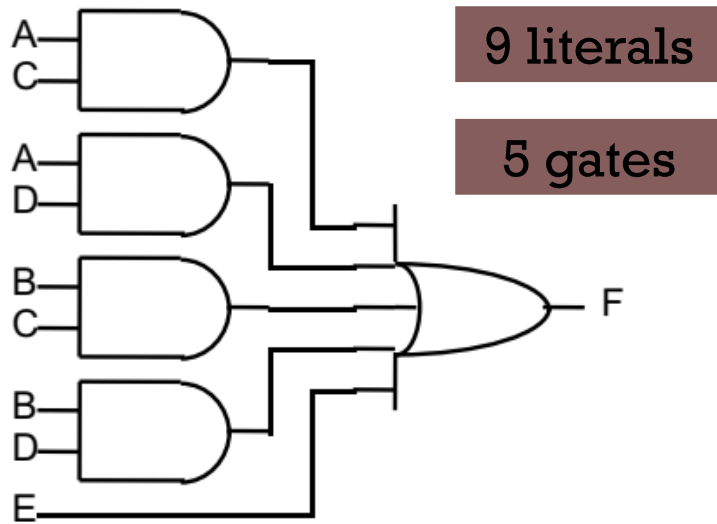


Node Manipulation

Factoring

Transformation of SOP-expressions in factored form.

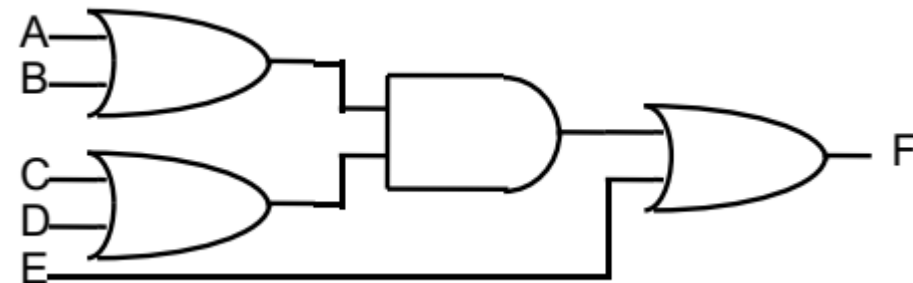
$$f = ac + ad + bc + bd + e \qquad f = (a + b)(c + d) + e$$



Before Factoring

9 literals

5 gates



5 literals

4 gates



Node Manipulation

Substitution

Substitution replaces an expression e within a function f with the value of an equivalent function $g(X) = e$.

$$f = (a + bc)(d + e)$$

$$f = g(d + e) \qquad g = a + bc$$



Node Manipulation

Collapsing

Also called elimination, collapsing is the reverse operation of the substitution.

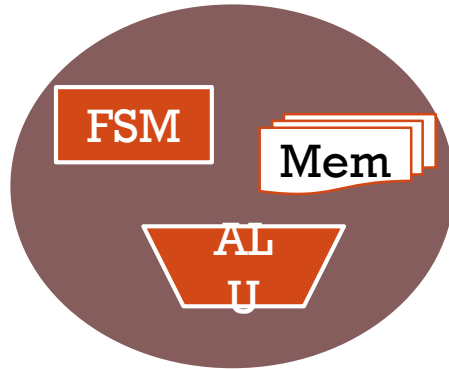
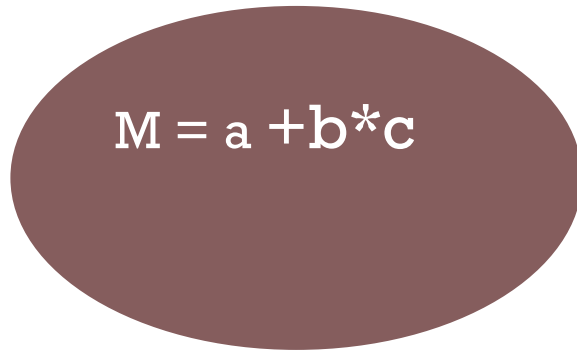
Collapsing is used to eliminate levels in order to meet the timing constraints.

$$f = ga + \bar{g}b \qquad g = c + d.$$

$$f = ac + ad + b\bar{c}\bar{d}$$

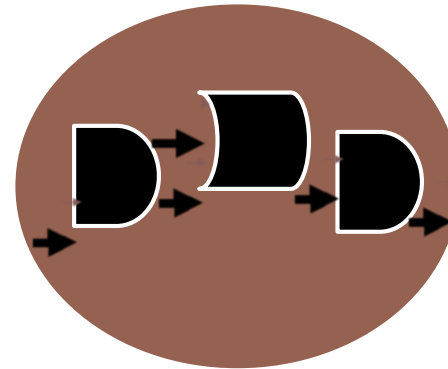
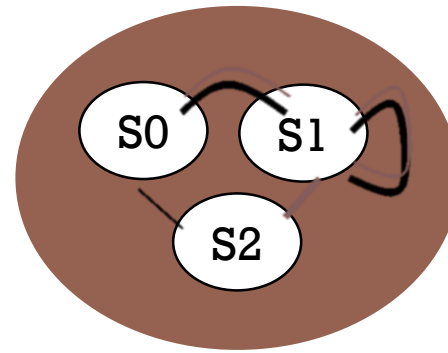


TAXONOMY OF SYNTHESIS

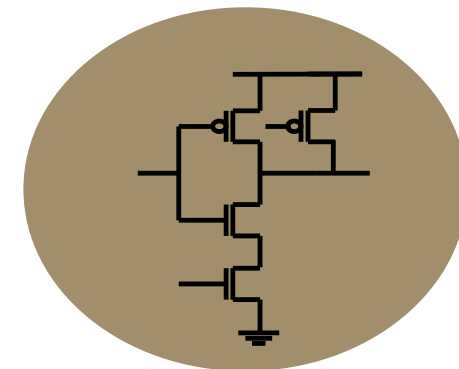
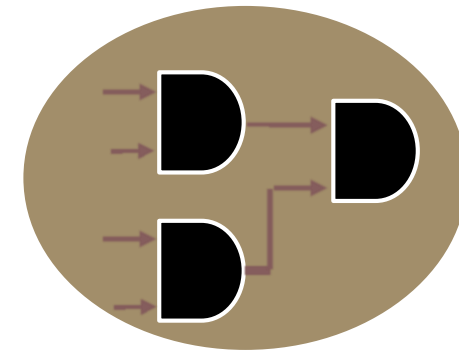


Architectural Synthesis

RC@Meetha V Shenoy



Logic Synthesis



Circuit Synthesis



LUT-based Technology Mapping

Technology-independent optimization phase ends with a reduced Boolean network

Next step → allocating LUTs for the implementation of the different nodes.



LUT-based Technology Mapping

Algorithms to minimize area

Chortle-crf

MIS-fpga

Xmap

Algorithms delay minimization

FlowMap

*Chortle-
d*

*DAG-
map*

*MIS-fpga-
delay*

Algorithms for maximizing routability

Other goals → testability and low power



LUT-based Technology Mapping

A formal definition of the problem that we intend to solve.

[LUT – Based Technology mapping problem]

Given a Boolean network representing a function f and an integer k . Find an implementation of f using only k – inputs LUTs, such that

The amount of LUTs use is minimal or

The delay of the resulting circuit is minimal.



Chortle-crf

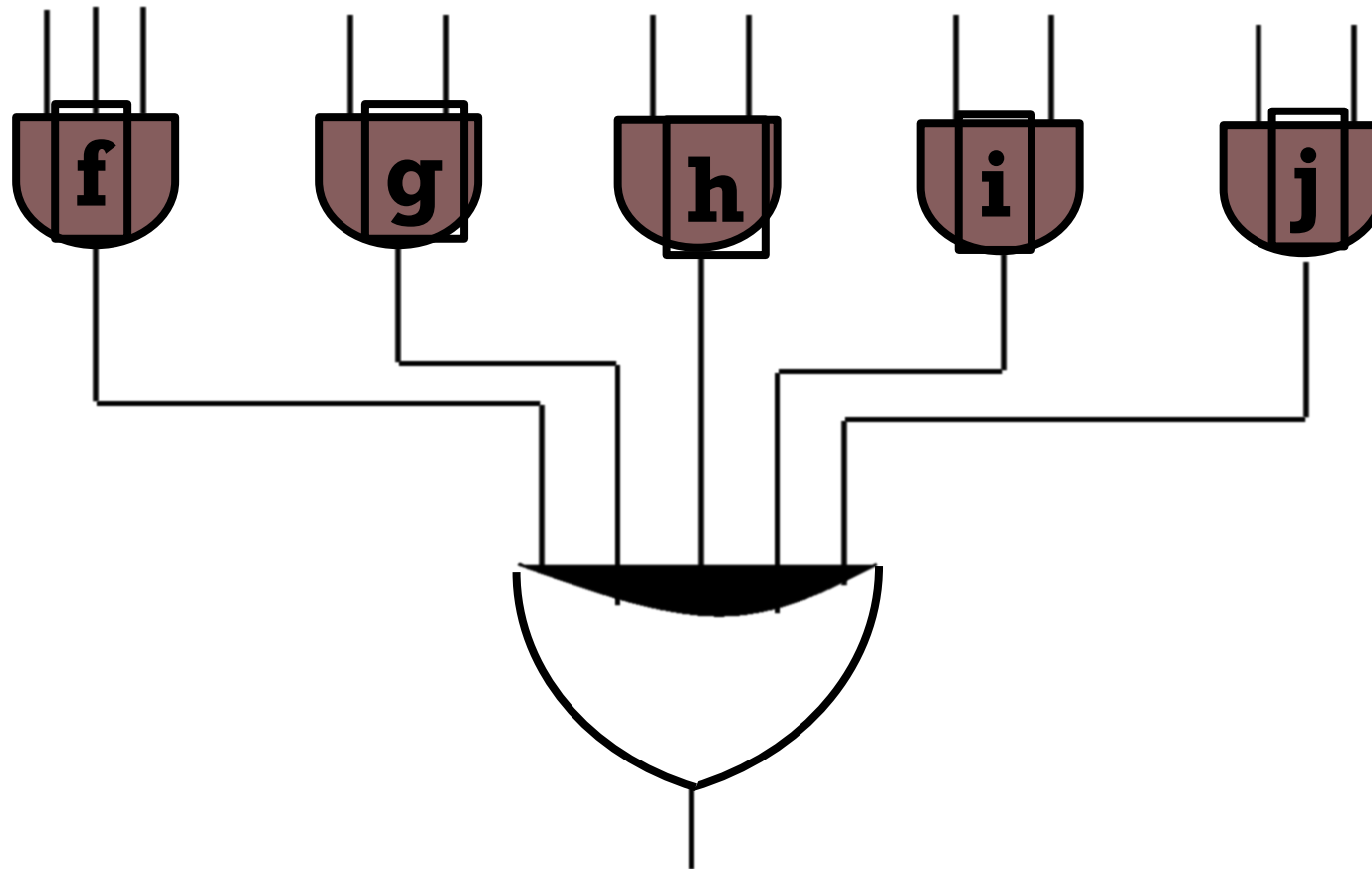
Implementation of Tree

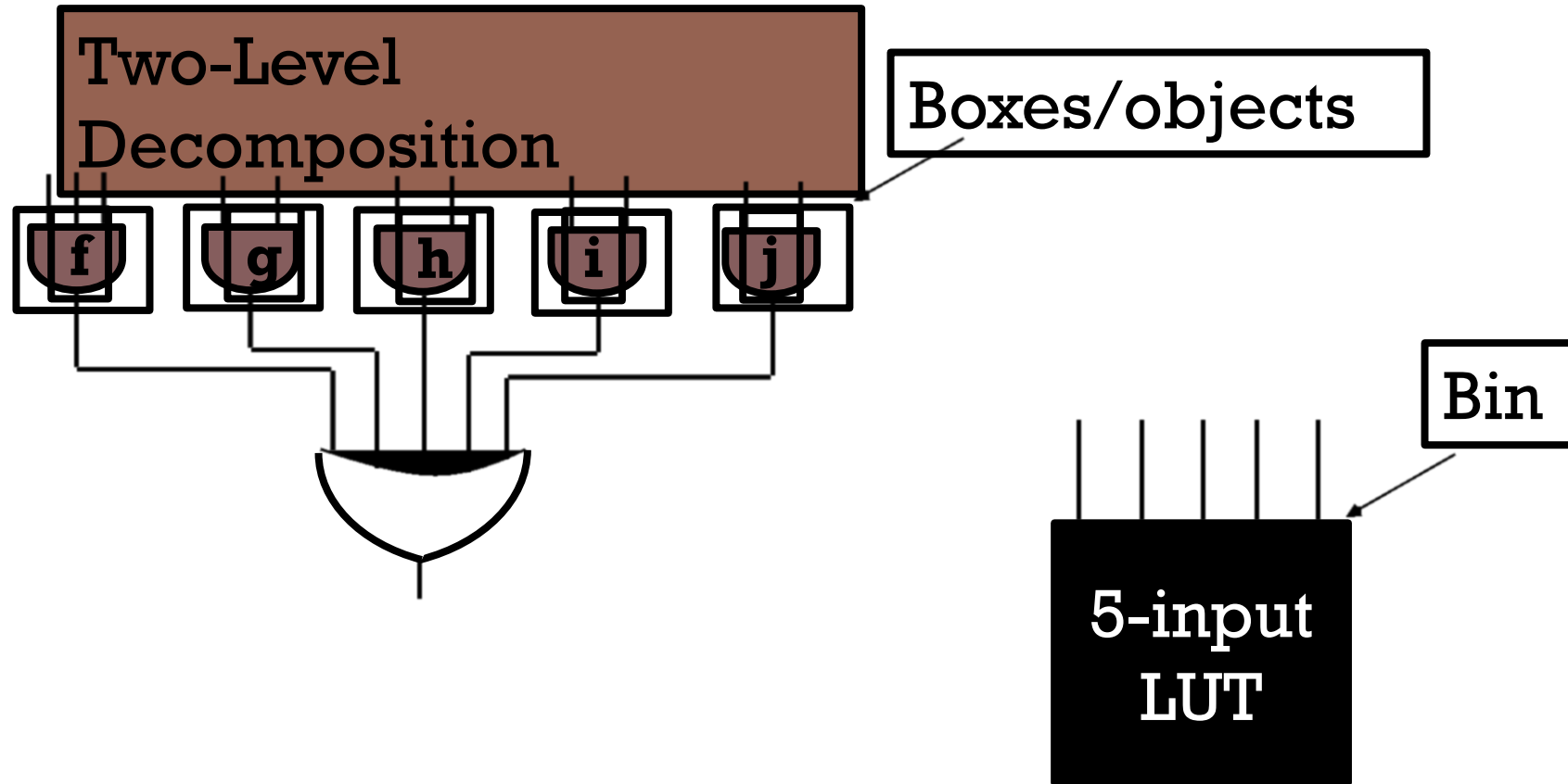
Two-Level Decomposition

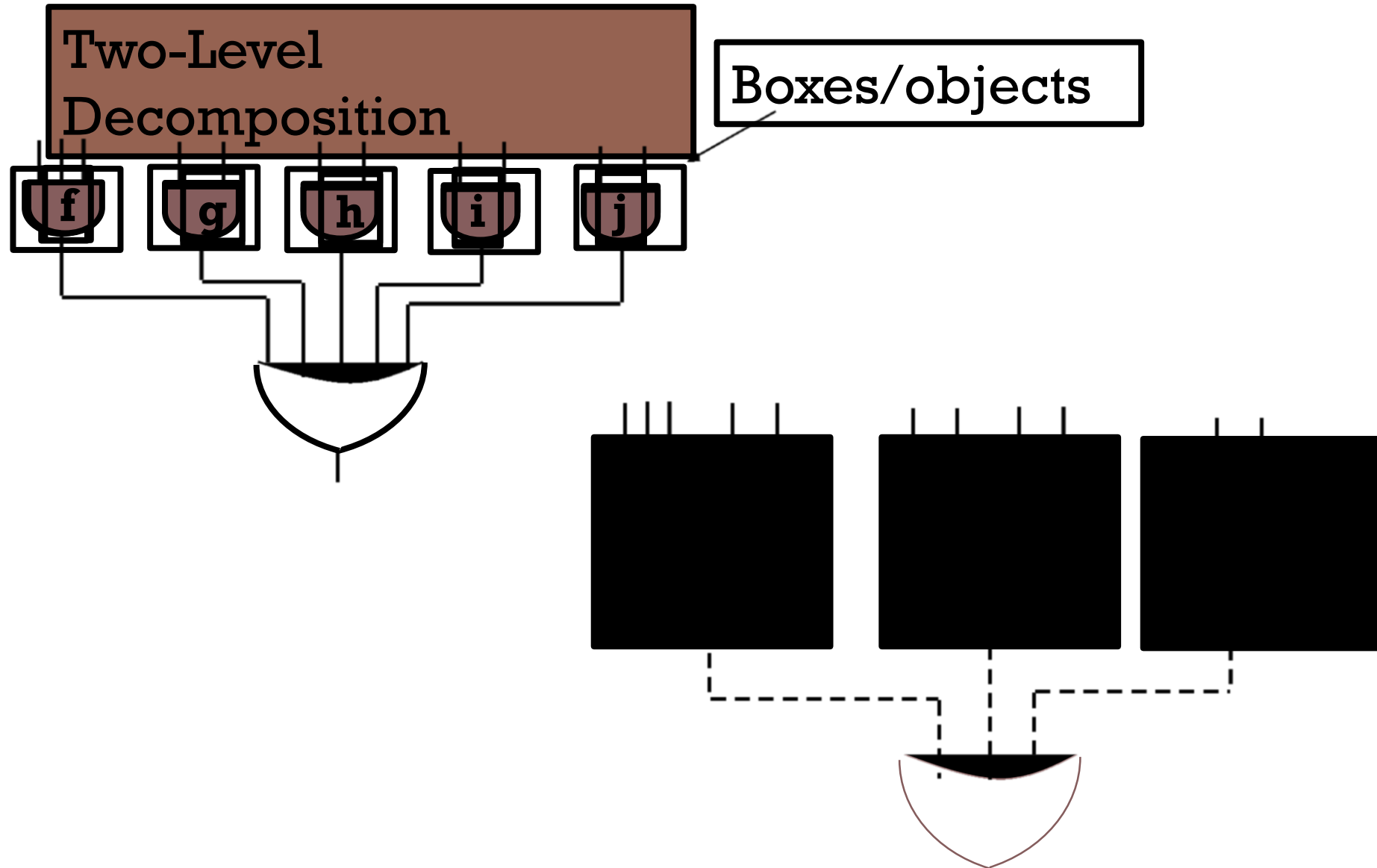
Multi-Level Decomposition



Two-Level Decomposition







Chortle-crf

Two-Level
Decomposition

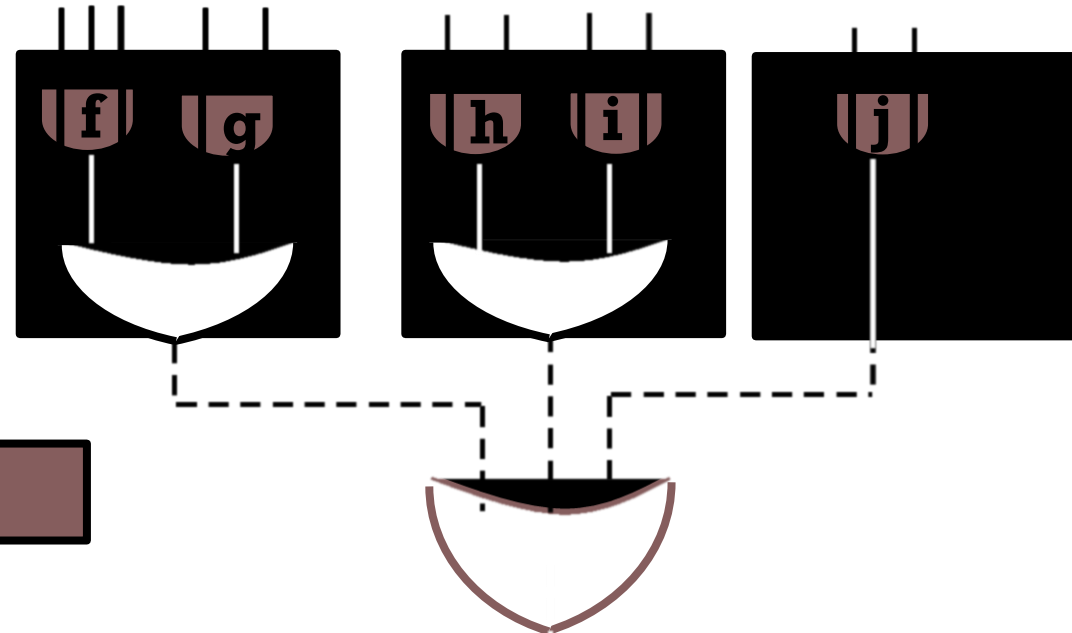
Boxes/objects



3, 2, 2, 2, 2

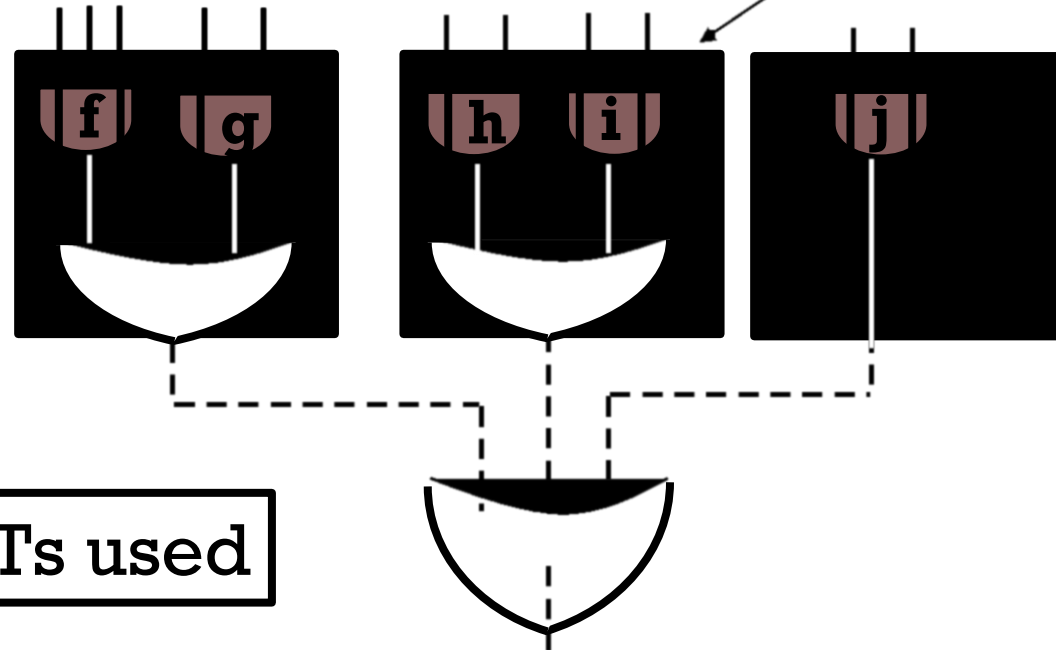
5, 4, 2

First Fit decreasing method



Chortle-crf

Multi-Level Decomposition

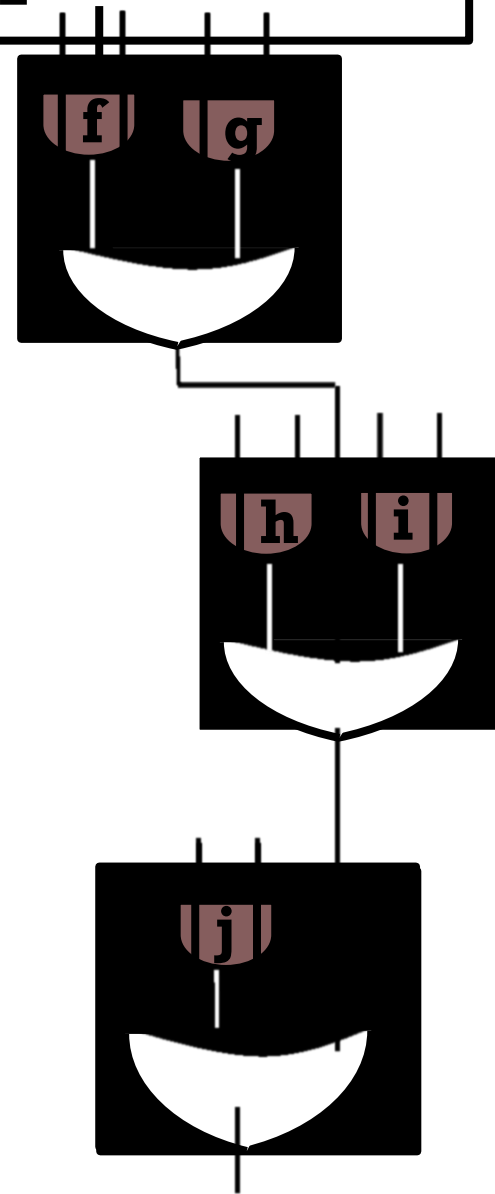
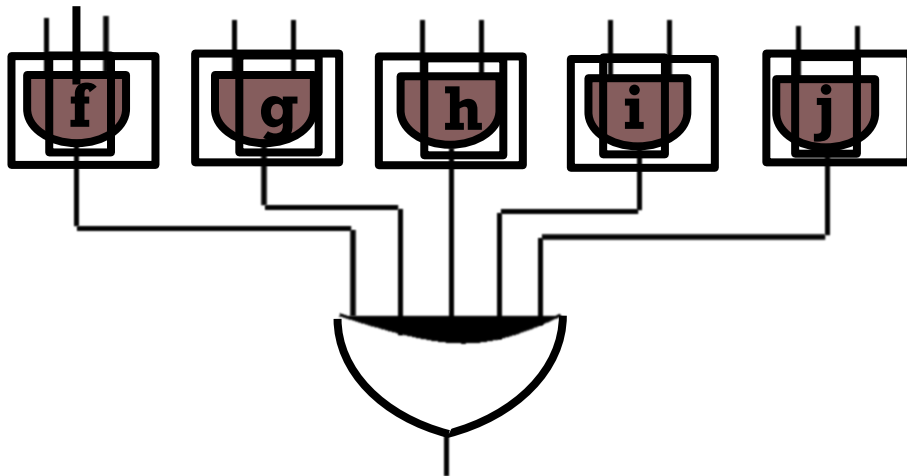


Tree of LUTs used

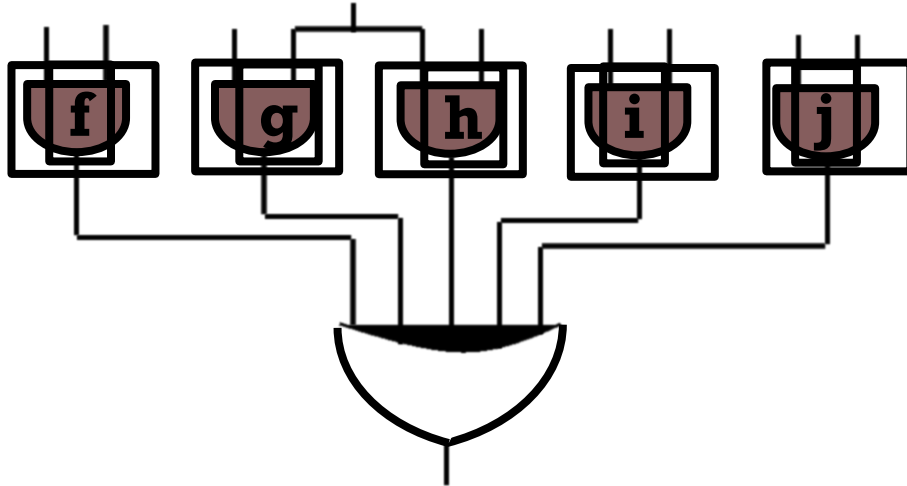
LUTs minimized by using unused pins



Chortle-crf

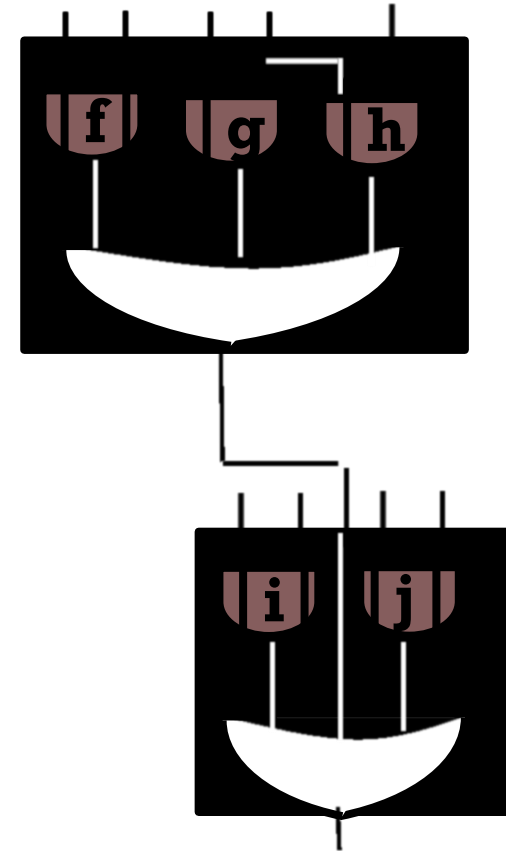
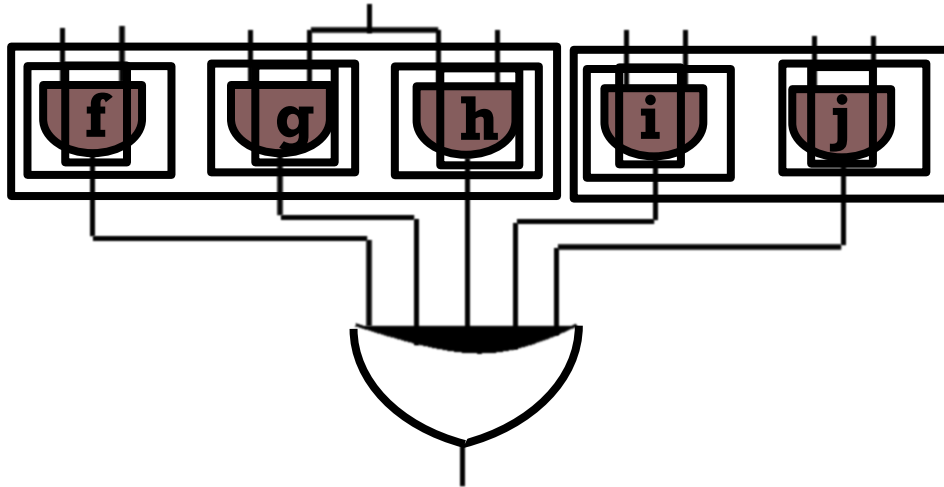


Exploiting Reconvergent Paths

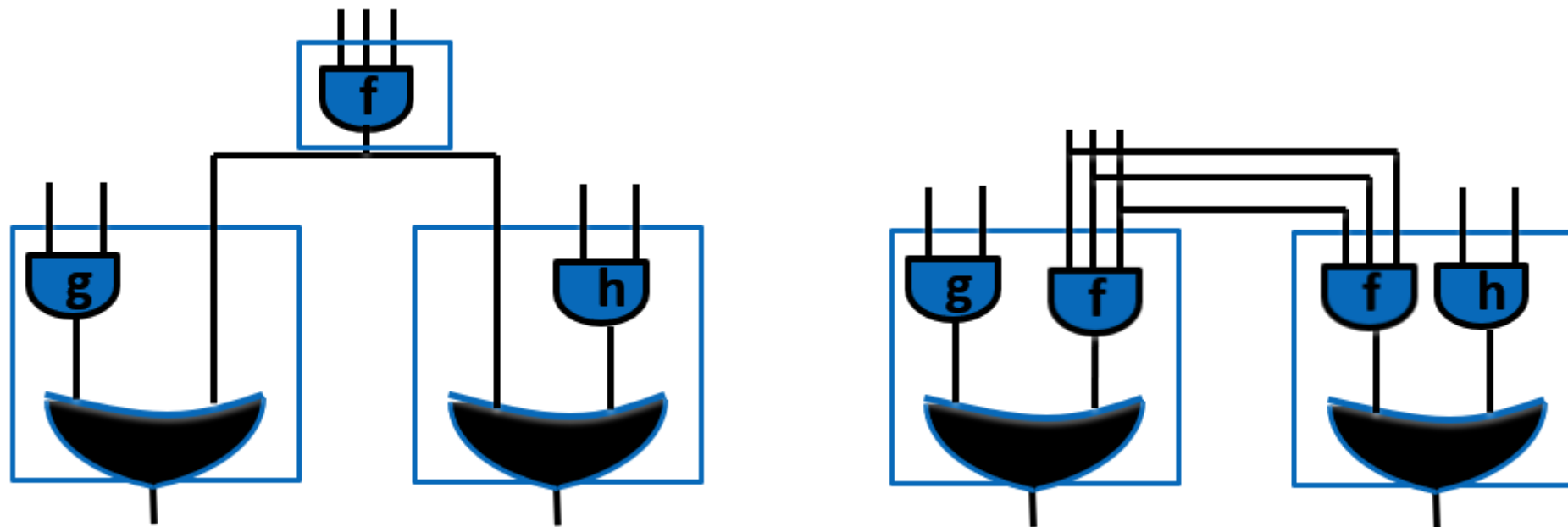


Chortle-crf

Exploiting Reconvergent Paths



Logic Replication



LUT-based Technology Mapping

Given a Boolean network G , we define the following:

A primary input (PI) node is a node without any predecessor.

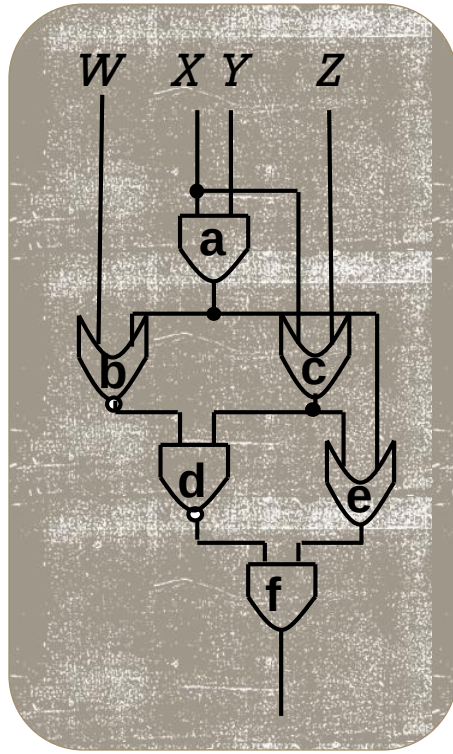
A primary output (PO) node is a node without any successor.

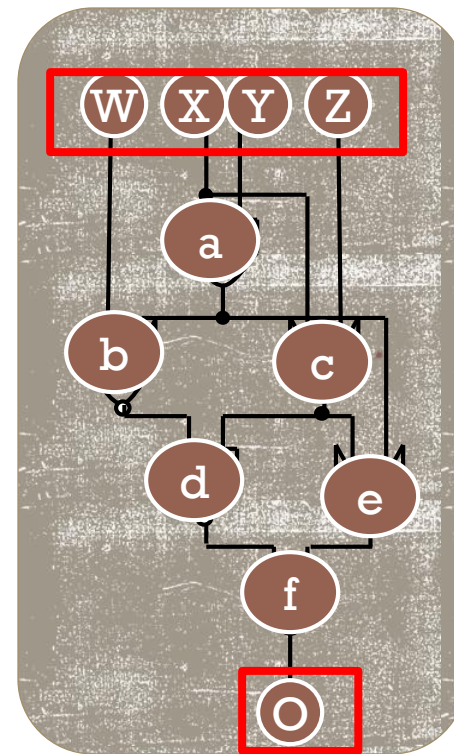
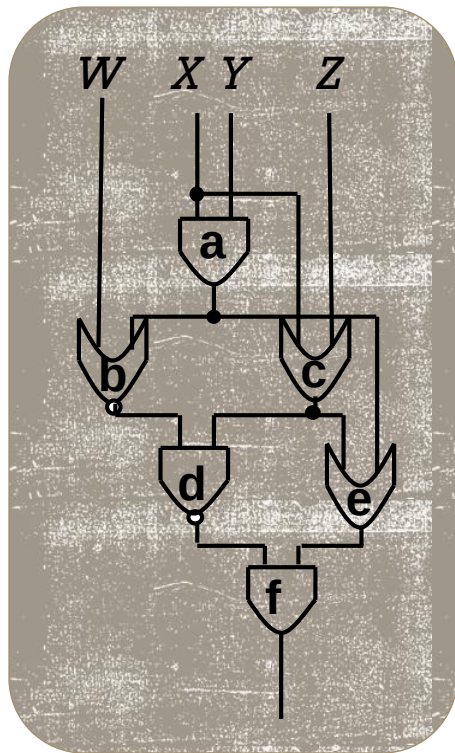
The level $l(v)$ of a node v is the length of the longest path from the primary inputs to v .

The depth of a network G is the largest level of a node in G .



LUT-based Technology Mapping





LUT-based Technology Mapping

Given a Boolean network G , we define the following:

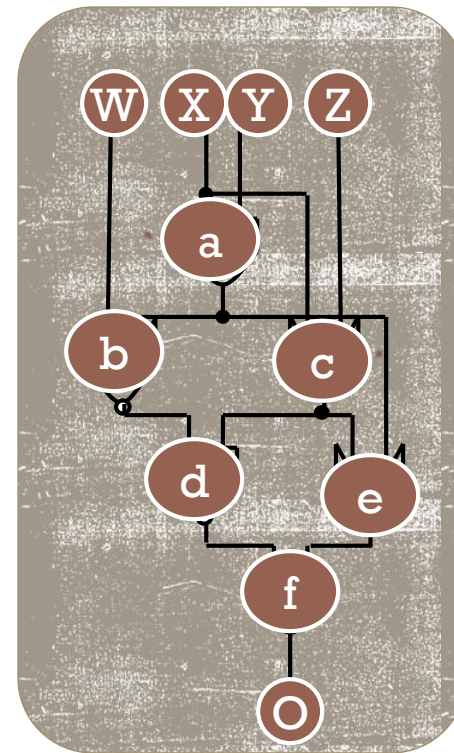
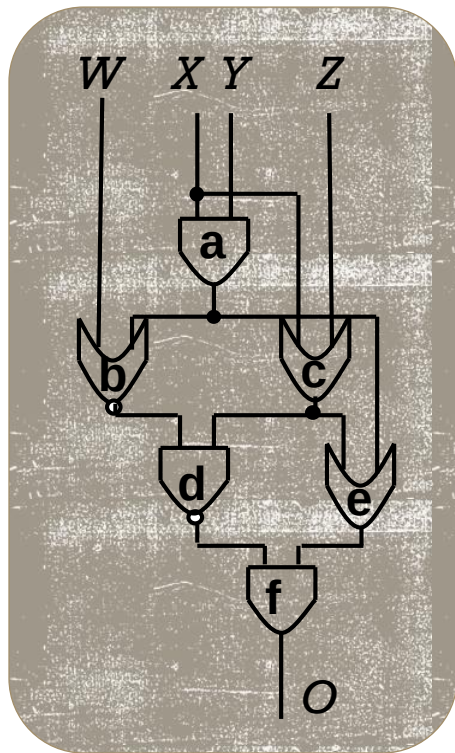
The fan-in of a gate v is the set of gates whose outputs are inputs of v .

Given a node $v \in G$, $\text{input}(v)$ is defined as the set of node of G , which are fan-in of v

The fan-out of v is the set of gates that use the output of v as input.

Given a node $v \in G$, $\text{output}(v)$ is defined as the set of node of G , which are fan-outs of v





LUT-based Technology Mapping

Given a Boolean network G , we define the following:

Given a subgraph H of a Boolean Network, $\text{input}(H)$ represent the set of distinct nodes outside H which supply inputs to gates in H

G is K -bounded if $|\text{input}(v)| \leq K$ for all nodes of G .

A tree or fan-out-free circuit is one in which each node has a maximal fan-out of one.



